LAB GUIDELINES

Lab 24 — Patch and Update Management

Maps to CompTIA Server+ objective **3.5** — Patching: testing, deployment, change management; and **2.4** — Proper patching procedures for clusters.

Run all commands on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): `unattended-upgrades`, `needrestart`, `apt-listchanges`, `debsecan`

```
apt update && apt install -y unattended-upgrades needrestart apt-listchanges debsecan
```

Step 1 — Inventory current package versions

```
dpkg -l | wc -l
apt list --upgradable 2>/dev/null | head -20
```

Capture as the **pre-patch baseline** in the change ticket from Lab 17.

Step 2 — Identify security-only updates

```
unattended-upgrade --dry-run -v 2>&1 | grep -E "Allowed origins|Packages" | head
debsecan --suite "$(lsb_release -cs)" --format detail 2>/dev/null | head -20 || true
apt-get -s dist-upgrade | grep -i security | head
```

The exam separates **security patches** (apply quickly) from **feature updates** (apply on a normal cadence).

Step 3 — Test in staging (snapshot first)

In a real fleet you snapshot the VM, apply patches there, and run a smoke test before touching production. Inside Killercoda, snapshot a LVM volume (Lab 3 pattern):

```
lvs 2>/dev/null
# Example: lvcreate -L 200M -s -n pre_patch_$(date +%F) /dev/vg_data/lv_root
```

Smoke-test plan template:

```
[ ] Service starts: systemctl is-active nginx mariadb ssh
[ ] HTTP 200 from key URL
[ ] DB SELECT 1 returns
[ ] No new errors in /var/log/syslog for 10 min
[ ] No regression in /var/log/nginx/access.log status codes
```

Step 4 — Apply patches with change-management discipline

```
apt update
DEBIAN_FRONTEND=noninteractive apt -y -o Dpkg::Options::="--force-confdef" \
    -o Dpkg::Options::="--force-confold" upgrade
```

Discipline rules:

1. **One change at a time** — apply patches **only** in the maintenance window.
2. **Notify impacted users** before the window opens (objective 4.1).
3. **Document** every package upgraded — `apt list --installed | diff -` against the pre-patch list.

Step 5 — needrestart: which services need a kick after upgrade

```
needrestart -b
```

Output flags services whose binaries / libraries changed but whose processes still hold the old code in memory.

Step 6 — Reboot decision: kernel and microcode

```
ls /boot/vmlinuz-* | sort
```

```
uname -r
[ -f /var/run/reboot-required ] && cat /var/run/reboot-required
```

If kernel, libc, systemd, or CPU microcode changed → reboot. Cluster-aware reboot pattern from Lab 14: drain node from LB, reboot, re-add.

Step 7 — Configure unattended-upgrades for security-only auto-apply

```
cat > /etc/apt/apt.conf.d/50unattended-upgrades <<'EOF'
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}ESMApps:${distro_codename}-apps-security";
    "${distro_id}ESM:${distro_codename}-infra-security";
};
Unattended-Upgrade::Automatic-Reboot "false";
Unattended-Upgrade::Mail "ops@lab.local";
EOF

cat > /etc/apt/apt.conf.d/20auto-upgrades <<'EOF'
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::AutocleanInterval "7";
EOF
unattended-upgrade --dry-run -d 2>&1 | tail
```

Server+ patching philosophy: **automate security**, **gate everything else** through change management.

Step 8 — Cluster-aware patching workflow recap (objective 2.4)

```
node1 → drain in LB (haproxy: disabled) → patch → reboot → smoke test → re-enable
node2 → repeat
node3 → repeat
```

Never patch all cluster nodes in parallel; you lose the redundancy you built.

Step 9 — Rollback plan

Server+ 4.1 says: "If the problem is not resolved, reverse the change". For packages:

```
# Pin a specific version (rollback)
apt install -y --allow-downgrades nginx=1.18.0-6ubuntu14.4
apt-mark hold nginx
apt-mark showhold
```

For wider rollback: revert to the pre-patch snapshot from step 3.

Step 10 — Patch report

```
{
  echo "Host:      $(hostname -f)"
  echo "Patched:   $(date -Is)"
  echo "Kernel:    $(uname -r)"
  echo "Reboot pending: $([ -f /var/run/reboot-required ] && echo YES || echo NO)"
  echo "--- Packages upgraded in this window ---"
  grep " upgrade " /var/log/apt/history.log | tail -50
} | tee /root/patch-report-$(date +%F).log
```

Attach to the change ticket and close.

Free online tools

- **Ubuntu Security Notices** — https://ubuntu.com/security/notices
- **Red Hat Security Advisories** — https://access.redhat.com/security/security-updates/
- **NVD / CVE database** — https://nvd.nist.gov/
- **CVSS calculator (FIRST)** — https://www.first.org/cvss/calculator/

What you learned

- Distinguishing security patches from feature updates.
- Snapshot-then-test workflow with a reusable smoke-test checklist.
- `needrestart` and reboot decision-making.
- Configuring unattended-upgrades for security-only auto-apply.
- Cluster-aware patching from objective 2.4.
- Rollback via package pinning or snapshot.
- A patch report you can attach to the change record from Lab 17.